

CA3T3: Design and Analysis of Algorithms

Unit 1

- Introduction: What is an Algorithm?
- Fundamentals of Algorithmic problem solving,
- Fundamentals of the Analysis of Algorithm Efficiency,
- Analysis Framework,
 - Measuring the input size,
 - Units for measuring Running time,
 - Orders of Growth,
 - Worst-case, Bestcase and Averagecase efficiencies.
- Asymptotic Notations and Basic Efficiency classes,
- Informal Introduction, O-notation, Ω -notation, θ -notation,
- mathematical analysis of non-recursive algorithms,
- mathematical analysis of recursive algorithms.

UNIT-1

1. Introduction

What is an Algorithm?

An algorithm is a finite sequence of well-defined, unambiguous instructions designed to solve a specific problem or perform a particular task. It takes one or more inputs, processes them according to predefined steps, and produces the desired output within a finite amount of time.

In simple terms, an algorithm is a step-by-step procedure for solving a problem.

Definition Algorithm: A finite set of instructions that, when executed, performs a particular task and terminates after producing the correct result.

Example: Algorithm to Find the Sum of Two Numbers

Input: Two numbers A and B

Output: Sum of A and B

Algorithm: sum of two numbers

1. Start
2. Read A and B.
3. Compute $\text{Sum} = A + B$.
4. Display Sum.
5. Stop.

Characteristics of an Algorithms:

A good algorithm should satisfy the following characteristics:

1. Input

An algorithm accepts zero or more inputs.

Inputs are supplied before execution.

Example: Two integers to be added.

2. Output

It produces at least one output.

The output should be meaningful and related to the given input.

Example: Sum of two integers.

3. Definiteness

Every instruction must be clear, precise, and unambiguous.

There should be no confusion during execution.

Example:

✓ Add A and B

✗ Process the numbers somehow

4. Finiteness

The algorithm must terminate after a finite number of steps.

It should never run forever.

5. Effectiveness

Every instruction must be simple enough that it can be performed in finite time.

Each step should be executable using available resources.

6. Correctness

The algorithm should produce the correct output for all valid inputs.

7. Generality

It should solve a class of problems rather than only one specific instance.

Properties of an Algorithm:

The important properties include:

1. Correctness

Produces the expected output for all valid inputs.

2. Efficiency

Uses minimum resources such as:

- Time
- Memory
- CPU

Efficiency is measured using:

- Time Complexity
- Space Complexity

3. Robustness:

Can handle unexpected or invalid inputs without crashing.

4. Simplicity:

Should be easy to understand, implement, and maintain.

5. Scalability:

Should work efficiently even when the input size becomes very large.

6. Modularity:

Can be divided into smaller reusable modules or functions.

7. Maintainability:

Easy to modify and improve in the future.

Algorithm vs Program:

Algorithm	Programs
Step-by-step solution to a problem	Implementation of an algorithm in a programming language
Language independent	Language dependent
Easier to understand	Requires programming knowledge
Written using natural language, pseudocode, or flowcharts	Written using languages like C, C++, Java, Python
Focuses on logic	Focuses on execution
Cannot be directly executed	Can be compiled/interpreted and executed
Independent of hardware	Depends on compiler/interpreter and hardware

Example:

Algorithm to Find the Sum of Two Numbers

Input: Two numbers A and B

Output: Sum of A and B

Algorithm:

1. Start
2. Read A and B.
3. Compute $\text{Sum} = A + B$.
4. Display Sum.
5. Stop.

Program (C)

```
#include<stdio.h>

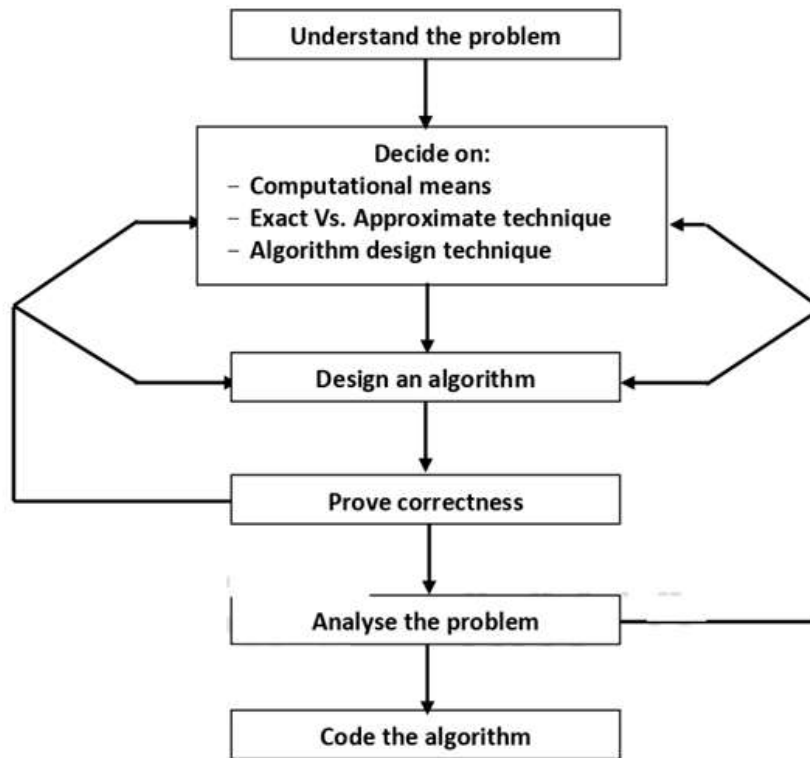
int main()
{
    int a,b,sum;
    scanf("%d%d",&a,&b);
    sum=a+b;
    printf("%d",sum);
    return 0;
}
```

2. Fundamentals of Algorithmic problem solving

This chapter introduces the systematic approach to solving computational problems. It explains how to analyze a problem, choose an appropriate algorithm design technique, and verify that the designed algorithm produces correct results.

Algorithm design and analysis process are:

1. Understanding the problem
2. Decide on Computational Means
3. Decide on Exact and Approximate Problem Solving
4. Decide on Algorithm Design Technique
5. Decide on Data Structures.
6. Methods of Specifying an Algorithm
7. Proving an Algorithm's correctness
8. Analyzing an Algorithm
9. Coding an Algorithm



Algorithm design and analysis process

Understanding Computational Problems:

The first and most important step in algorithm design is to clearly understand the problem that needs to be solved.

Without understanding the problem, it is impossible to develop a correct algorithm.

Definition: A computational problem is a problem that can be solved by a computer using a sequence of well-defined instructions (an algorithm). The objective is to transform given input into the required output efficiently.

Objectives

- Identify the inputs.
- Identify the expected outputs.
- Understand all constraints.
- Determine special cases.
- Understand user requirements.

A computational problem generally consists of:

- **Input:** Data provided to the algorithm.
- **Output:** Expected result after processing.
- **Constraints:** Conditions or limitations (such as time, memory, or input size).

Example

Problem: Find the largest number in an array.

Input: An array of integers:

$A = \{12, 25, 8, 40, 15\}$

Output: 40

Decide on Computational Means

Computational means refer to selecting the computing resources and environment required to solve the problem.

Includes

- Computer Hardware
- Software
- Programming Language
- Memory
- Processing Speed

Example:

Large Data Processing Need:

- High RAM
- Multiple processors
- Cloud Computing

Decide on Exact and Approximate Problem Solving

Some problems require an exact answer, while others can be solved using an approximate answer.

In short: Produces the correct answer every time.

Examples

- Sorting
- Searching
- Banking Software
- Payroll
- Library Management

Example

- $10 + 20 = 30$
- No approximation is allowed.

Approximate Solution

Produces a near-optimal solution.

Used when exact solutions are too expensive or time-consuming.

Examples

- Route Optimization
- Artificial Intelligence
- Machine Learning
- Weather Forecast
- Image Recognition

Decide on Algorithm Design Technique

An Algorithm Design Technique is a strategy used to construct an algorithm efficiently.

Different problems require different techniques.

Common Techniques

1. **Brute Force:** Try every possible solution.
Example: Linear Search
2. **Divide and Conquer:** Divide problem into smaller sub-problems.
Example:
 - Merge Sort
 - Quick Sort
 - Binary Search

3. **Greedy Method:** Choose the best option at every step.

Examples

- Prim's Algorithm
- Kruskal's Algorithm
- Dijkstra's Algorithm

4. **Dynamic Programming:** Store previous results.

Examples

- Fibonacci
- Knapsack
- Matrix Chain Multiplication

5. **Backtracking:** Try solutions, Undo if wrong.

Examples

- N Queen
- Sudoku

6. **Branch and Bound:** Used for optimization.

Examples

- Traveling Salesman Problem
- Job Assignment

Decide on Data Structures

A Data Structure organizes data efficiently so algorithms can process it quickly.

Importance

Correct data structures improve

- Speed
- Memory usage
- Simplicity

Common Data Structures are: Array, Linked List, Stack, Queue, Trees, Graphs

Methods of Specifying an Algorithm

An algorithm can be represented in several ways.

- Natural Language: Written in plain English.
- Pseudocode: Uses English mixed with programming keywords.
- Flowchart
- Graphical representation
- Programming Language

Proving an Algorithm's Correctness

Algorithm correctness means proving that the algorithm always produces the correct output for every valid input.

Methods

- Mathematical Proof: Uses induction.
- Loop Invariant: Shows loop correctness.
- Proof by Contradiction: Assumes incorrectness, Shows contradiction.

Example: Finding Largest Number.

Analyzing an Algorithm

Algorithm analysis measures the efficiency of an algorithm before implementation.

Two Types

- Time Complexity: Measures execution time.
- Space Complexity: Measures memory usage.

Why Analyze?

- Compare algorithms
- Reduce execution time
- Reduce memory usage
- Improve performance

Coding an Algorithm

Coding is the process of converting the algorithm into a programming language.

Steps:

Step 1: Choose programming language, such as C, C++, Java, Python, etc.

Step 2: Translate algorithm.

Step 3: Compile.

Step 4: Debug errors.

Step 5: Test using different inputs.

Step 6: Optimize if necessary.

Components of a Computational Problem:

1. Input

The data provided to the algorithm.

Example:

$A = \{5, 9, 3, 7\}$

2. Output

The desired result after computation.

Example:

Maximum = 9

3. Constraints

Restrictions that the algorithm must satisfy.

Examples:

- Maximum input size
- Available memory
- Execution time
- Input range

Example:

$1 \leq N \leq 10^6$

4. Objective

Clearly defines what the algorithm should accomplish.

Example:

- Sort an array.
- Find the shortest path.
- Search for an element.

Steps in Problem Solving:

Problem solving is a systematic process.

Step 1: Understand the Problem

Identify:

- Input
- Output
- Constraints
- Objectives

Example: Find the maximum element in an array.

Step 2: Analyze the Problem

Determine:

- Possible approaches
- Available resources
- Complexity requirements

Step 3: Design the Algorithm

Write the algorithm using:

- Pseudocode
- Flowchart

Choose appropriate:

- Data structures
- Algorithmic technique

Step 4: Verify Correctness

Perform:

- Dry run
- Boundary testing
- Sample testing

Step 5: Analyze Complexity

Evaluate:

- Time Complexity
- Space Complexity

Step 6: Implement

Convert/ Translate the algorithm into a programming language.

Examples:

C

C++

Java

Python

Step 7: Test and Debug

Check:

- Syntax errors
- Logical errors
- Runtime errors

Step 8: Optimize

Improve:

- Execution time
- Memory usage
- Code readability

Algorithm Design Techniques /Types of Algorithms:

Algorithms are classified based on the technique used to solve problems.

1. Brute Force Algorithm

- Simplest approach.
- Tries every possible solution.

Examples:

- Linear Search
- Bubble Sort
- Selection Sort

Advantages

- Easy to understand
- Always finds the solution

Disadvantages

- Slow for large inputs

2. Divide and Conquer

Break the problem into smaller subproblems, solve them recursively, and combine the results.

Examples

- Merge Sort
- Quick Sort
- Binary Search

Advantages

- Efficient
- Faster execution

3. Greedy Algorithm

Makes the best local choice at each step, aiming for a globally optimal solution.

Examples

- Kruskal's Algorithm
- Prim's Algorithm
- Dijkstra's Algorithm
- Huffman Coding

Advantages

- Fast
- Easy implementation

Disadvantages

- Does not always guarantee the optimal solution.

4. Dynamic Programming

Solves overlapping subproblems and stores their results to avoid repeated computations.

Examples

- Fibonacci Series

- Knapsack Problem
- Matrix Chain Multiplication
- Longest Common Subsequence

Advantages

- Reduces computation time
- Efficient for optimization problems

5. Backtracking

Builds solutions incrementally and abandons a path when it cannot lead to a valid solution.

Examples

- N-Queens Problem
- Sudoku Solver
- Rat in a Maze
- Graph Coloring

6. Branch and Bound

Explores the solution space while pruning branches that cannot produce a better solution.

Examples

- Traveling Salesman Problem
- Job Scheduling

7. Randomized Algorithm

Uses random numbers or probabilistic decisions during execution.

Examples

- Randomized Quick Sort
- Monte Carlo Algorithms

8. Recursive Algorithm

Calls itself repeatedly until a base condition is reached.

Examples

- Factorial
- Fibonacci
- Tower of Hanoi

9. Iterative Algorithm

Uses loops instead of recursion.

Examples

- Iterative Binary Search
- Iterative Factorial

10. Searching Algorithms

Used to locate an element in a collection.

Examples

- Linear Search
- Binary Search

11. Sorting Algorithms

Arrange data in a specific order.

Examples

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort

Correctness of Algorithms:

An algorithm is correct if it produces the expected output for every valid input and terminates after a finite number of steps.

Correctness ensures that:

- The algorithm solves the intended problem.
- The output is always accurate.
- The algorithm terminates.

Requirements for Correctness

1. Correct Input Handling The algorithm should accept all valid inputs.
2. Correct Processing Every step should follow the intended logic.
3. Correct Output The output should satisfy the problem requirements.
4. Termination The algorithm must finish after a finite number of steps.

Methods to Verify Correctness

1. Dry Run
Execute the algorithm manually with sample inputs.
2. Boundary Testing
Test minimum and maximum valid input values.
3. Test Cases
Check normal, invalid, and edge-case inputs.
4. Mathematical Proof
Prove correctness using formal methods such as:
 - Loop Invariants: A condition that remains true before and after each iteration of a loop.
 - Mathematical Induction: Proves recursive algorithms or repeated processes by showing the base case is true and that if the algorithm works for one case, it also works for the next.

Example: Correctness of Finding the Maximum Element

Algorithm:

1. Set **max** = A[0].
2. Compare each remaining element with **max**.
3. If an element is greater, update **max**.
4. After the last comparison, output **max**.

Input:

A = {10, 25, 8, 40, 15}

Execution:

$\text{max} = 10$
 $25 > 10 \rightarrow \text{max} = 25$
 $8 < 25 \rightarrow \text{max} = 25$
 $40 > 25 \rightarrow \text{max} = 40$
 $15 < 40 \rightarrow \text{max} = 40$

Output:

Maximum = 40

Since every element is compared and the largest value is retained, the algorithm always returns the correct maximum.

Key Differences: Verification vs Validation

Verification	Validation
Checks whether the algorithm is logically correct	Checks whether it solves the intended real-world problem
Performed during algorithm design	Performed through testing with real inputs
Focuses on correctness	Focuses on user requirements

3. Fundamentals of the Analysis of Algorithm Efficiency

Algorithm efficiency analysis is the study of how much time and memory an algorithm requires as the input size grows. It helps compare algorithms and choose the most suitable one for a problem.

1. Why Analyze Algorithms?

Two algorithms may solve the same problem but differ significantly in performance.

Example:

- Algorithm A sorts 1,000 items in 1 second.
- Algorithm B sorts 1,000,000 items in 5 seconds.

Analysis helps predict performance for large inputs without actually running the program.

2. Input Size (n)

The efficiency of an algorithm is usually expressed as a function of the input size n .

Examples:

- Number of elements in an array
- Number of nodes in a graph
- Number of digits in an integer

3. Time Efficiency

Time efficiency measures the number of basic operations executed by an algorithm.

Example

for $i = 1$ to n
 print(i)

The statement print(i) executes **n times**.

Therefore:

$$T(n) = n$$

Time complexity: $O(n)$

4. Space Efficiency

Space efficiency measures the amount of memory required.

Example

```
int array[n]
```

The memory required grows proportionally with n.

Space complexity: $O(n)$

5. Types of Analysis

- **Best-Case Analysis:** Minimum time required.
Example: Linear search finds the item in the first position.
 $T(n) = 1$
- **Worst-Case Analysis:** Maximum time required.
Example: Linear search checks every element.
 $T(n) = n$
- **Average-Case Analysis:** Expected running time over all possible inputs.

6. Asymptotic Analysis

Asymptotic analysis studies algorithm performance as n becomes very large.

- **Big-O Notation (Upper Bound)**

Represents the worst-case growth rate.

Examples:

Complexity	Name
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Linearithmic
$O(n^2)$	Quadratic
$O(2^n)$	Exponential
$O(n!)$	Factorial

- **Big-Ω (Omega) Notation**

Represents the lower bound (best-case growth).

Example:

$$T(n) = 3n + 2$$

$$\Omega(n)$$

- **Big-Θ (Theta) Notation**

Represents the exact asymptotic growth.

Example:

$$T(n) = 5n + 10$$

$$\Theta(n)$$

7. Principles of Efficient Algorithms

1. Minimize unnecessary computations.
2. Reduce nested loops when possible.
3. Use appropriate data structures.
4. Prefer logarithmic or linear solutions over quadratic ones.
5. Consider both time and space requirements.

4. Analysis Framework:

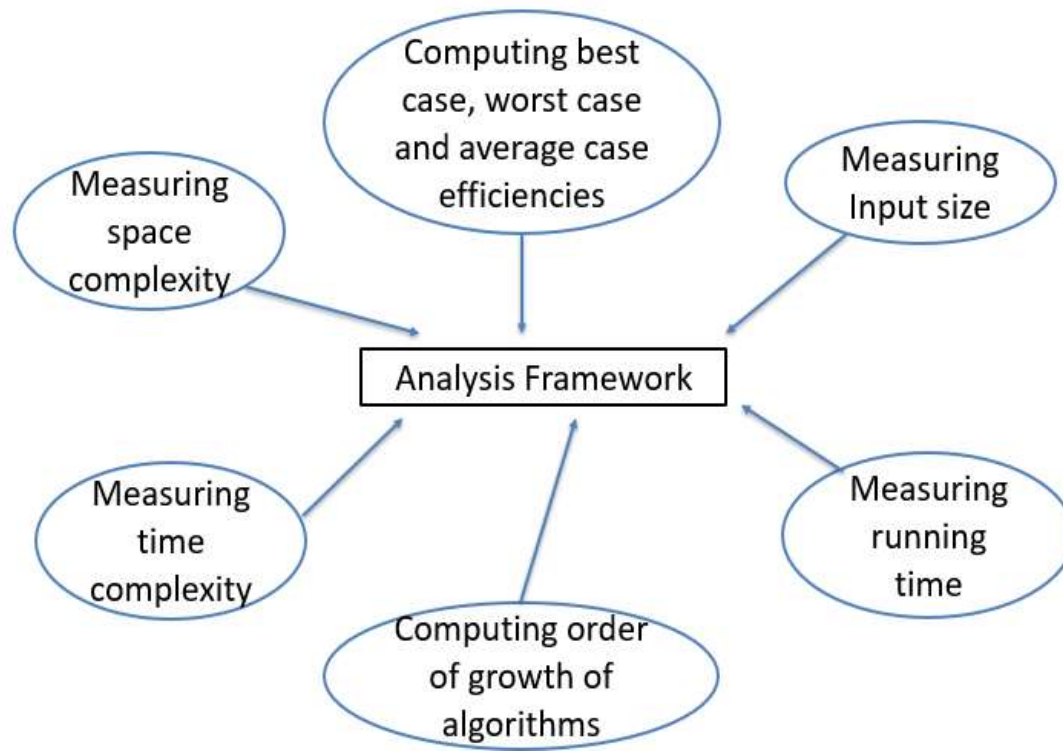
There are two kinds of efficiency:

Time Efficiency

Time efficiency, also called time complexity of an algorithm is the amount of computer time it needs to run to completion.

Space Efficiency

Space efficiency, also called space complexity of an algorithm is the amount of memory it needs to run to completion.



General framework for analyzing the efficiency of algorithms is:

Algorithm analysis is the process of finding **how fast an algorithm works** and **how much memory it uses** before implementing it.

The efficiency of an algorithm depends on several factors.

1. Measuring an Input's Size
2. Units for Measuring Running Time
3. Orders of Growth
4. Worst-case, Best-case and Average-case Efficiencies.

Measuring the input size

- How can we measure an input size? For each input there is a different aspect to size.
- The **input size** is the amount of data that an algorithm receives.
- We usually represent the input size by **n**.
- The larger the input, the more work the algorithm usually performs.
- Why Measure Input Size?
- Input size helps us estimate: Running time, Memory usage, Performance, Efficiency
- Different problems measure input size differently.
- There is no single method for every algorithm.
- Sometimes the input size depends on **how the data is stored**.
- Sometimes input size depends on what operation is performed.

- Sometimes the algorithm works on a **single object**, and the input size depends on its value.
- For some algorithms (especially cryptography and number theory), input size is measured by the number of **bits** required to represent the number in binary.
- Input size is influenced by.
 - i. The Data Representation: e.g. Matrix
 - ii. Operations of the algorithm: e.g. Spell checker. Input size is the number of characters in the string
 - iii. Properties of objects: e.g: checking for prime or not
- Input size is measured by the number of bits in the n 's binary representation: $b = \log_2 n + 1$

Units for measuring Running time

- Different units for measuring an algorithm's running time are:
 - i. Measuring using standard time unit: in seconds, milliseconds. - Does not work as different architectures have the different computing speeds.
 - ii. Count number of times each operation is executes
- Best approach is to find the number of times the algorithms basic operation is executed on inputs of size n .
- How to find: it is usually the most time-consuming operation in the algorithm's innermost loop.
- To find out how to compute such a count for non-recursion algorithms.

Orders of Growth

- We cannot compare one algorithm with other algorithms with small inputs.
- Measuring the performance of an algorithm in relation with the input size ' n ' is called order of growth.
- The order of growth tells, how the program would behave as the order of size of input is increased.

n	$\text{Log } n$	$n \log n$	n^2	2^n
1	0	0	1	1
2	1	2	4	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65,536
32	5	160	1024	4,294,967,296

Worst-case, Bestcase and Averagecase efficiencies.

The running time of an algorithm increases as the input size(n) increases. Sometimes even if the size of the input is same, the running time varies among different instances of the input.

In that case, we perform best, average and worst-case analysis.

1. Worst-case efficiency:

If an algorithm takes maximum amount of time to run to completion for a specific set of input then it is called worst case time complexity.

2. Best-case Efficiency:

If an algorithm takes minimum amount of time to run to completion for a specific set of input then it is called best case time complexity.

3. Average-case Efficiency:

The time complexity that we get for certain set of inputs is as a average same. Then for corresponding input such a time complexity is called average case time complexity.

Example: Sequential (Linear) Search Algorithm

Find Best, Worst, and Average Case Analysis

Algorithm: Sequential (Linear) Search

Sequential search checks each element one by one until the required element is found or the list ends.

Algorithm: SequentialSearch(A[1...n], key)

```

for i ← 1 to n do
    if A[i] = key then
        return i
return -1

```

Where:

A = Array of n elements

key = Element to be searched

1. Best Case

Condition

The key is found in the **first position**

Example:

Search key = **10**

Comparisons:

10	20	30	40	50	60	70
----	----	----	----	----	----	----

$10 == 10 \checkmark$

Number of comparisons = **1**

Time Complexity

$$T(n) = 1$$

Therefore,

$$\boxed{\text{Best Case} = O(1)}$$

2. Worst Case

Condition

The key is

- at the **last position**, or
- **not present** in the array.

Example 1:

Search key = **70**

Comparisons:

10	20	30	40	50	60	70
----	----	----	----	----	----	----

Total comparisons = **7**

Example 2

Search key = 80

Comparisons: 10.....70

Not found after checking all elements.

Total comparisons = 7

Time Complexity

$$T(n) = n$$

Therefore,

Worst Case = $O(n)$

3. Average Case**Condition**

The key is equally likely to be present at any position.

Number of comparisons:

Position	Comparisons
1	1
2	2
3	3
4	4
5	5
6	6
7	7

Average comparisons

$$\begin{aligned}
 &= \frac{1 + 2 + 3 + 4 + 5 + 6 + 7}{7} \\
 &= \frac{28}{7} \\
 &= 4
 \end{aligned}$$

General formula

$$\frac{1 + 2 + 3 + \dots + n}{n}$$

Using

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Therefore,

$$\text{Average Comparisons} = \frac{n(n+1)}{2n} = \frac{n+1}{2}$$

Hence,

$$T(n) = \frac{n+1}{2}$$

Ignoring constants,

$$\text{Average Case} = O(n)$$

- **Best Case:** Element found at the first position → **1 comparison** → **$O(1)$**
- **Worst Case:** Element found at the last position or absent → **n comparisons** → **$O(n)$**
- **Average Case:** Element equally likely at any position → **$(n + 1)/2$ comparisons** → **$O(n)$**

Case	Comparisons	Time Complexity
Best Case	1	$O(1)$
Average Case	$(n + 1)/2$	$O(n)$
Worst Case	n	$O(n)$

5. Asymptotic Notations and Basic Efficiency classes

Informal Introduction, O -notation, Ω -notation, θ -notation,

Basic Efficiency classes:

Class	Name	Comment
1	Constant	Best case efficiency
$\log n$	Logarithmic	Ex: Binary Search, Quick sort(worst-case) Problem's size reduces by constant factor
n	Linear	Ex: Bubble sort(Best case), Shell sort(Best case), Insertion sort(Best case), Heap sort(worst case), {Algorithm with 1 loop}
$n \log n$	"n-log-n"	Ex: M, Quick sort, Merge sort, Heap sort, Many divide and conquer algorithm
n^2	Quadratic	Ex: Selection sort, insertion sort, bubble sort {Algorithm with 2 embedded loops}
n^3	Cubic	Ex: Matrix multiplication, Gaussian elimination {Algorithm with 3 embedded loops}
2^n	Exponential	Ex: Travelling salesman, Knapsack, Combinatorial objects. Algorithm generating all subsets of n -element set
$n!$	Factorial	Algorithm that generate all permutations of n -element set.

Asymptotic notations are:

1. Big Oh(O-notation)
2. Omega(Ω -notation)
3. Theta (θ -notation)

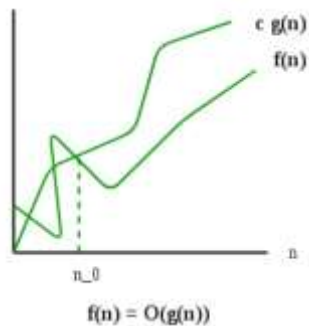
Big Oh(O-notation)

- The Big oh notation is denoted by 'O'.
- It is a method of representing the upper bound of algorithms running time.
- Using big oh notation we can give longest amount of time taken by the algorithm to complete.

Definition

Let $f(n)$ and $g(n)$ be two non-negative functions.

Let n_0 and constant C are two integers such that n_0 denotes some value of input and $n > n_0$. Similarly C is some constant such that $C > 0$. We can write $f(n) \leq C * g(n)$.

Graph Representation

$$f(n) = O(g(n)) \text{ or } f(n) \leq c * (g(n))$$

Omega(Ω -notation)

- Omega notation is denoted by ' Ω '.
- This notation is used to represent the Lower bound of algorithms running time.
- Using omega notation we can denote shortest amount of time taken by algorithm.

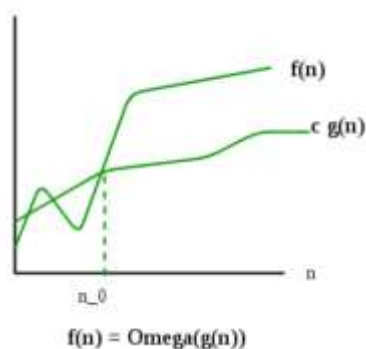
Definition

A function $f(n)$ is said to be in $\Omega(g(n))$ if $f(n)$ is bounded below by some positive constant multiple of $g(n)$ such that.

$$f(n) \geq C * g(n) \quad \text{For all } n \geq n_0$$

Graph Representation

$$f(n) \geq C * g(n) \quad \text{or } f(n) = \Omega(g(n))$$



Theta(θ -notation)

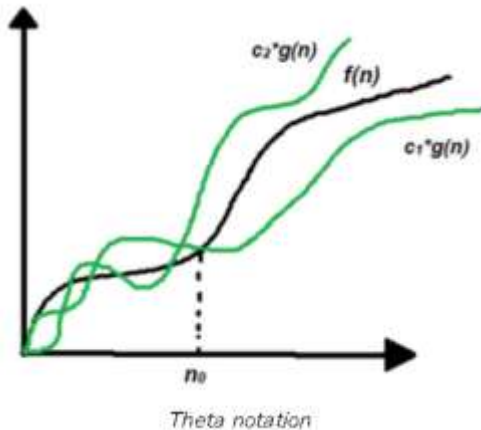
- The theta notation is denoted by θ .
- By this method the running time is between upper bound and lower bound.

Definition

Let $f(n)$ and $g(n)$ be two non-negative functions. There are two positive constants namely C_1 and C_2 such that.

Graph Representation

$$C_2 * g(n) \leq f(n) \leq C_1 * g(n)$$

**Mathematical analysis of non-recursive algorithms****General plan for analyzing efficiency of Non-recursive algorithms:**

1. **Decide on parameter 'n'** indicating input size.
2. **Identify basic operation** of algorithm [it is located in the inner-most loop.]
3. **If the number of executions of the basic operation** depends only on input size; then, find the worst case, average case, and best case efficiency also.
4. **Setup sum expression** $C(n)$ reflecting number of times basic operation is executed.
5. **Simplify summation** using standard and rules to establish order of growth.

Example: Find the largest element in an Array**Algorithm**

Algorithm: MaxElement($A[0 \dots n-1]$)

Input: Array $A[0 \dots n-1]$ of real numbers

Output: Largest element in A

Begin

$\text{max} = A[0]$

 for $i = 1$ to $n-1$ do

 if $A[i] > \text{max}$ then

$\text{max} = A[i]$

 return max

End

Step 1: Decide the Input Size Parameter

The input size is the **number of elements** in the array.

n = Number of elements in the array

Step 2: Identify the Basic Operation**The comparison**

$$A[i] > \max$$

is the basic operation because it is executed in every iteration of the loop. The assignment $\max = A[i]$ is executed only when a larger element is found, so it is not the basic operation.

Step 3: Best, Average, and Worst Cases

For this algorithm, the number of comparisons depends **only on the size of the array (n) and not on the arrangement of elements.**

Case	Number of Comparisons
Best Case	(n-1)
Average Case	(n-1)
Worst Case	(n-1)

Although the number of assignments to max varies with the input order, the number of comparisons is always the same. Therefore, all three cases have identical time complexity.

Step 4: Set Up the Summation

The comparison is executed once during each iteration of the loop.

The loop runs from, **i=1 to n-1.**

Therefore,

$$C(n) = \sum_{i=1}^{n-1} 1$$

Step 5: Simplify the Summation

Using the summation rule,

$$\sum_{i=1}^{n-1} 1 = n - 1$$

Hence,

$$C(n) = n - 1$$

Order of Growth

Ignoring constants,

$$T(n) = \Theta(n)$$

Therefore,

- Time Complexity: $\Theta(n)$ (Linear Time)
- Best Case: $\Theta(n)$
- Average Case: $\Theta(n)$
- Worst Case: $\Theta(n)$

Space Complexity

The algorithm uses only one extra variable (max).

$$\text{Space Complexity} = \Theta(1)$$

(Constant Space)

In Short Answer:

- Input Size: n
- Basic Operation: Comparison ($A[i] > \text{max}$)
- Number of Comparisons: $n - 1$
- Best = Average = Worst Case: $\Theta(n)$
- Space Complexity: $\Theta(1)$

Mathematical analysis of recursive algorithms.

Recursive algorithms solve a problem by calling themselves on smaller instances of the same problem. Their efficiency is analyzed using recurrence relations instead of simple loop counting.

General Plan for Analyzing Recursive Algorithms:

1. **Decide on a parameter** input size ' n '.
2. Identify the algorithm's **basic operation**.
3. **Find number of times basic operation is executed** on different inputs of the same size.
If it varies; then
Find the worst case, average case and best case efficiency also.
4. Set up a **recurrence relation** with an appropriate initial condition.
5. Solve the **recurrence to obtain the order of growth** of its solution.

Example: Find the factorial of n

Problem: Find the factorial of a given number.

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

Example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Algorithm:

Algorithm: factorial(n)

Input: Integer n

Output: Factorial of n

if ($n == 0$ or $n == 1$)

return 1

else

return $n * \text{factorial}(n-1)$

Optional: Recursion Tree**N=5**

Example for factorial (5)

factorial (5)

 $5 \times \text{factorial}(4)$ $4 \times \text{factorial}(3)$ $3 \times \text{factorial}(2)$ $2 \times \text{factorial}(1)$

1

Returns

1

↑

 $2 \times 1 = 2$

↑

 $3 \times 2 = 6$

↑

 $4 \times 6 = 24$

↑

 $5 \times 24 = 120$ **Solution:****Mathematical Analysis****Step 1: Input Size**Input size parameter: n **Step 2: Basic Operation**

Basic operation is Multiplication

 $n * \text{factorial}(n-1)$ **Step 3: Count Basic Operations**

For, factorial(5)

Multiplications performed: $5 \times 4 \times 3 \times 2 \times$

Total multiplications

$$4 = n - 1$$

Hence,

$$C(n) = n - 1$$

Best Case = Worst Case = Average Case

since every execution performs exactly **$n-1$ multiplications.****Step 4: Recurrence Relation**

Each recursive call performs:

- One recursive call
- One multiplication

Therefore,

$$T(n) = T(n - 1) + 1$$

Initial condition

$$T(1) = 1 \text{ (or } T(0) = 1)$$

Step 5: Solve the Recurrence

Using substitution,

$$\begin{aligned}T(n) &= T(n-1) + 1 \\&= T(n-2) + 2 \\&= T(n-3) + 3 \\&\vdots \\&= T(1) + (n-1)\end{aligned}$$

Since

$$\begin{aligned}T(1) &= 1 \\T(n) &= 1 + (n-1) \\T(n) &= n\end{aligned}$$

Therefore,

$$T(n) = \Theta(n)$$

Unit-1, Unit Wise Previous Year Question Papers Compulsory Questions in Exam

2 Marks

1. What is Algorithm/Define Algorithms.
2. Define an Algorithm. Mention the characteristics of an algorithm.
3. Write any 5 characteristics of an algorithm.
4. Define order of growth.
5. What is Recursive Algorithm?
6. Define Time Complexity.
7. What is Space Complexity?
8. What is best case, average case and worst case.
9. Mention the best case, average case and worst case time complexities of linear search algorithm.
10. Arrange the following complexities in ascending order:
 $O(n \log n)$ $O(n^3)$ $O(n)$ $O(n^2)$ $O(1)$
11. State the different efficiency classes.

5 Marks

1. Explain the fundamentals of algorithmic problem solving.
2. What are the criteria that an algorithm must satisfy?
3. Explain the characteristics of an algorithm.
4. Explain Algorithm design and analysis process with flow diagram.
5. Write an algorithm to find the largest element in an array and also perform mathematical analysis.
6. Write an algorithm to find the factorial of a number using recursion and also perform mathematical analysis.
7. Obtain the time complexity of the code below:

```
a = 0;
for i = 1 to n do
  for j = 1 to n - i do
    a = a + i * j;
```

8. Explain time and space complexity of an algorithm with an example.
9. Explain asymptotic notations Big O and Big Θ that are used to compare the order of growth of an algorithm with example.
10. Write a note on Asymptotic Notations.
11. Give general plan of mathematical analysis of recursive algorithm with example.
12. Write an algorithm using iterative function to find sum of n numbers.

10 Marks

1. Explain the asymptotic notations in brief or Illustrate asymptotic notations with examples.
2. Explain the Analysis Framework in detail.
3. Compare the best, worst and average case complexity with suitable example.
4. Explain the fundamentals of algorithm problem solving in detail.
5. What are the algorithm design techniques? Explain.
6. Explain mathematical analysis of recursive algorithm and non- recursive algorithm